Chapter 6: Conclusions, Recommendations and Future Works

6.1 Introduction

This chapter closes the chain from the problem stated in Chapter 1 to the evidence reported in Chapter 5. The problem was that small language models that run privately on consumer devices know far less than large cloud models, and that a frozen small model cannot be retrained cheaply to close the difference. The study built CAMR, an external memory engine that sits beside a frozen small model. It measured how much of the capability gap the engine closes, for which kinds of task, at what token and time cost, and for which model sizes. It then packaged the engine as an application. Section 6.2 states what the evidence established, section 6.3 what should be done because of it, and section 6.4 what remains open. No new results are introduced; each statement refers to the section of Chapter 5 that supports it.

6.2 Conclusions

The conclusions answer the research questions of section 1.4 in order.

6.2.1 Research Question i: Techniques and Their Limits On-Device

The literature review (Chapter 2) identified similarity retrieval, recency and importance scoring, compression and graph-based association as the main memory techniques, all designed for large models. Implementing them for a small model on a CPU showed which of them transfer. Recency and importance (Equation 4.1) lowered full-support recall on both multi-hop benchmarks, because benchmark questions are independent and recency carries no signal about them (section 5.8.1). Two capability-adaptive additions transferred well. Gating kept irrelevant text away from a weak reader. Entity bridging, a one-step associative hop that needs no graph construction, raised full-support recall on 2WikiMultiHopQA from 35.8% to 78.4% at the same token budget, twice the recall of standard top-10 RAG with fewer tokens. Compared head to head with standard RAG (section 5.8.3), CAMR matched or beat it when the answer was stored, with about 40% less text, but its similarity gate failed exactly as RAG did when the store held notes about *similar* entities and not the right one; an entity-anchored gate, validated on questions that played no part in its design, removed most of that failure at no cost on answerable questions. The applicable techniques on-device are therefore cheap, selective and anchored in what the question names, not the weighted composites of large-model agent memories, and not retrieval alone.

6.2.2 Research Question ii: How Much of the Gap a Frozen-Weights Memory Engine Closes

Against the frontier cloud model (Kimi K3), the engine closed the entire single-hop knowledge gap for a 0.5B model (gap closed 1.00) and half of the multi-hop gap (0.50, 95% CI 0.20–0.90). The multi-hop result was confirmed on 200 held-out questions with the configuration fixed in advance. With CAMR, a 3B model reached 54.0% exact match against the cloud model's closed-book 51.8%, at 5.8 s instead of 18.4 s per answer, and the gain from memory excluded zero for every small model tested (section 5.8.11). Across 19 models, 18 reached or exceeded the cloud model on long-tail facts once they had memory (section 5.8.10). Fine-tuning the same 0.5B model on the same documents, the alternative the problem statement argued against, reached 13.3% against memory's 71.7% on the same held-out questions, after a 3.5-hour search; memory stored the documents in 6 s (section 5.8.13).

6.2.3 Research Question iii: Knowledge-Bound Versus Reasoning-Bound Tasks

Yes. Memory helps knowledge-bound tasks and does not help reasoning-bound ones. The residual gap between a 0.5B and a 20B model reading the same notes was 10.0 points on single-hop facts, 23.3 on multi-hop questions and 83.3 on mathematical reasoning (section 5.8.4). Most of the remaining multi-hop gap is a reading gap: the small model had the answer in its context for 76% of its misses (section 5.8.6). Worked-example memory for mathematics helped some model families and hurt others, depending on family rather than size.

6.2.4 Research Question iv: The Token Budget That Maximises Gap Closed Per Token

For a 0.5B model the best budget was about 128 tokens for single facts and 512 for multi-hop questions. Larger budgets lowered accuracy because extra notes distracted the small model, and reasoning tasks did not benefit at any budget (section 5.8.8). Retrieval recall saturated near 1,024 tokens (section 5.8.1). Tuning the read path on validation questions confirmed the trade-off from the other side: judged on retrieval alone, a 1,024-token budget recovers the most evidence, but only a strong reader can use it (section 5.8.2). The best budget therefore depends on the task and on the model, which is why a learned policy was needed. Tuned by cross-validation on training questions only, the policy matched the cloud model's accuracy (66.7% against 63.9%, a difference within sampling noise) while 75% of questions stayed on the device and mean time per answer fell from 12.6 s to 2.9 s (section 5.8.12).

6.2.5 Research Question v: Cost in Tokens and Latency on a Consumer Device

The engine added a median of 187 ms and about 536 prompt tokens per answer for a 0.5B model. Peak memory was 712 MB for the engine and 1,987 MB for the model runtime, and the store was 47.8 MB (section 5.8.17). The ten-per-cent retrieval-latency target (NFR-02) was not met for the 0.5B model, because that model answers in about 180 ms. The overhead was diagnosed as CPU contention rather than engine cost: about 29 ms in isolation. Decode speed fell by 3–17% with memory, the cost of reading a longer prompt. On this class of device, models of 7B parameters or more with memory answered more slowly than the cloud model, which makes **1.5–4B the model size that suits the engine** on a CPU-only laptop. In the released application, loading the model in the background before the first question brought the first word to 0.6–1.7 s for 0.5–3B models, and a 21-fold larger memory (282 to 5,976 notes) added only about 10 ms to the lookup (section 5.8.18).

6.2.6 Beyond the Research Questions: a Model That Learns As It Goes

The last extension asked whether a small model with frozen weights can keep learning, and whether it then competes with larger models. It can, through its memory. Starting from an **empty** memory and looking up what it did not know, a 0.5B model answered 43.5% of 200 new questions, against 10.0% for the same model closed-book, 19.0% for a 3B model and 20.5% for a 20B model whose process needed 13.2 GB of RAM against 0.7 GB; offline and asked in other words, it kept 41.5%, and a 3B model reading the memory the 0.5B model had learned reached 45.0% (section 5.8.14). The most literal form of *external weights* was measured too: a token-level memory beside the frozen model (kNN-LM) reached 13.3% on held-out questions, the same as fine-tuning, against 75.0% for notes the model reads (section 5.8.15). Knowledge therefore grows best as text the model reads, not as numbers inside or beside it. Where the engine tried to learn its own retrieval, the evidence was negative: a query adapter never beat the untrained one on unseen questions, which is why sleep keeps one only when held-out recall rises (section 5.8.16). In the application, a personal question memory cannot answer is now met with a request to be taught rather than a bare *I don't know*, without any loss on the cases that guard against over-refusal (section 5.8.14), and nothing personal is ever sent to the web.

6.2.7 Contribution and the Boundaries of the Findings

The study contributes six things: (1) a working, open-source memory engine and evaluation harness, distinguished from standard RAG by measured, capability-adaptive decisions (gating, entity anchoring, bridging, per-question budgets and escalation); (2) measured

evidence that external memory lets small on-device models match a frontier cloud model on knowledge-bound questions, together with the residual gap that memory cannot close; (3) a size map of 19 small models showing where memory pays off; (4) a measured comparison with fine-tuning on the same documents; (5) a released desktop application, CAMR Personal, that puts the approach in users' hands; and (6) eager learning, knowledge packs and sleep around a frozen model, with privacy rules, measured against closed-book models of several sizes, fine-tuning and a token-level memory. These conclusions hold for English benchmark questions, for frozen quantised models run with greedy decoding on 4-core CPUs with 16 GB of memory, for the sample sizes reported (30–200 questions per condition, 500 for retrieval), and for a stochastic cloud reference sampled once. They do not yet extend to real users' personal data, to phones, or to sessioned conversations where recency matters. Table 6.1 summarises the alignment from objectives to conclusions.

Table 6.1: Alignment from objectives to evidence and conclusions

Objective (section 1.3.2)	**Evidence (Chapter 5)**	**Conclusion**	**Status**
i. Investigate capability gaps and memory techniques	Literature review; every proposed technique built switchable and measured (5.8.1, 5.8.4)	Similarity + gating + bridging transfer; Eq. 4.1 does not; the gap is mostly knowledge and reading	Met
ii. Design a lightweight memory engine (write policy, ranking, token budget)	Write path, screener, single-file store, adaptive read path, budgeter (5.4); eager learning, packs, sleep (5.4.11); TC-01–TC-58; D-01–D-23	Implemented and verified; ranking changed on evidence; distilled notes not evaluated at scale	Met, with two recorded deviations (5.8.19)
iii. Develop it on a consumer laptop without a GPU, with an embedded store	SQLite + sqlite-vec, CPU-only (5.2); device profile (5.8.17); app latency (5.8.18)	Runs in under 3 GB on 4 cores; NFR-02 not met for 0.5B (contention)	Met on laptop-class containers, not one physical laptop (5.8.19)
iv. Evaluate gap closed by task type, with tokens and latency	Four-condition harness; 19-model sweep; n = 200 (5.8.4, 5.8.10, 5.8.11)	Facts: fully closed; multi-hop: half, matched by 3B; reasoning: not closed	Met
v. Determine the best token budget	Budget sweep; learned policy (5.8.8, 5.8.12)	128 tokens (facts), 512 (multi-hop) for 0.5B; tuned routing matches the cloud with 75% on-device	Met

6.3 Recommendations

The recommendations are ordered by priority. The first three are essential safeguards for anyone deploying the approach; the rest are improvements. Each one names its audience and the evidence behind it.

Table 6.2: Recommendations

#	**Audience**	**Recommended action**	**Evidence and condition**
1	Developers deploying on-device assistants	Pair a 1.5–4B model with external memory for personal and long-tail facts;	5.8.10: facts alone ≤ 33%, with memory 70–80%; 1.5–4B faster than the cloud. Holds on

#	Audience	Recommended action	Evidence and condition
		do not rely on the small model's own knowledge	CPU-only devices
2	Developers deploying on-device assistants	Route reasoning-heavy questions (e.g. arithmetic) to a stronger model, with the user's consent, instead of adding memory	5.8.4: residual gap 83 points on GSM8K; 5.8.12: the learned router sent all maths to the cloud
3	CAMR maintainers	Code-sign the Windows and macOS downloads before wider distribution, and keep feedback strictly opt-in	5.6: unsigned builds trigger OS warnings; privacy is a stated design requirement (NFR-06)
4	Designers of memory engines for small models	Use similarity ranking with gating, entity bridging and an entity-anchored abstain check; do not rely on a similarity threshold alone to detect that the answer is not stored; avoid recency/importance weighting for independent questions; calibrate the abstain threshold per embedder	5.8.1: Eq. 4.1 lowered recall; bridging doubled 2Wiki recall; 5.8.3: entity gate; 5.8.2 calibration rule
5	Designers of memory engines for small models	Keep the memory budget small for tiny models (≈128 tokens for facts) and let a policy adapt it	5.8.8 budget curve; 5.8.12 learned policy
6	CAMR maintainers	Partition CPU threads between the model and the embedder on 4-core devices	5.8.17: p90 retrieval 157 → 50 ms with partitioning
7	Researchers evaluating memory for small models	Report the residual gap against a model reading the same notes, use a frontier ceiling, and use at least 200 questions for headline claims	5.8.4 (gap closed exceeds 1 against a weak ceiling); 5.8.11 (pilot vs n = 200)
8	Developers of local assistant apps	Load the model before the first question and keep it resident; stream answers and report time to the first word; keep memory budgets small for small models; partition CPU cores	5.8.18: warm-up removes the model load from the first answer; 5.8.8: smaller budgets are also more accurate for 0.5B
9	The student and supervisor	Obtain ethics approval and a consent statement before collecting feedback from real users of CAMR Personal	5.5.3: no user study yet; feedback export is designed to be metrics-only by default
10	Developers of local assistant apps	When memory has nothing for a general question, look it up, store what was read with its source, and answer from it; never send questions about the user, and ask to be taught instead; identify the client and honour rate limits	5.8.14: 43.5% vs 10.0% for the same model; 5.5.7; D-18, D-21
11	Designers of memory engines	Train any learned retrieval component only behind a held-out acceptance test, and prefer notes the model reads to weights inside or beside it	5.8.16: adapter never beat the identity on unseen questions; 5.8.15: kNN-LM = LoRA = 13.3% vs 75.0%

6.4 Future Works

The directions below extend the research. Every objective of the approved proposal was addressed; the points where the work departed from it (verbatim rather than distilled notes in the experiments, and laptop-class containers rather than one physical laptop) are recorded in section 5.8.19 and appear below where they leave a question open. The directions are ordered by expected value, and each follows from a limitation or open question in section 5.8.

Table 6.3: Prioritised future works

Priority	Direction and rationale	What changes / resources	How to evaluate
1	**Longitudinal user study of CAMR Personal.** Whether the assistant gets more useful as a person's own memory grows is the application's central claim, and no user has tested it yet (5.8.20).	Real users and their own notes over 4–8 weeks; ethics approval; the existing opt-in metrics export	Weekly share of answers drawn from memory, 👍 rate, and accuracy on each user's repeated questions (day 1 vs week 4)
2	**Sessioned memory benchmark.** Recency could not show a benefit on independent questions (5.8.1); conversations are not independent.	A sessioned benchmark such as LongMemEval or LoCoMo; the recency term already implemented	Gap closed with and without recency on the same sessions
3	**Closing the reading gap.** 76% of multi-hop misses had the answer in context (5.8.6).	Structured notes, answer-extraction prompts, or a small reader-tuned model, keeping the engine fixed	Correct-when-in-context on HotpotQA and 2Wiki, n ≥ 200
4	**CAMR on phones.** Porting to phones was delimited out of this study (1.7.2), but it is the natural home of an assistant that grows with its user. A starting point exists: an Android prototype (Kotlin port of the read and write paths, MediaPipe embedder, Compose interface, 57 MB arm64 package) compiles and passes 6 unit tests, including token counting identical to the Python engine. It has never run on a device.	Two to three mid-range Android phones; on-device model files; gating calibration for the phone embedder	Retrieval share of latency (NFR-02), peak memory, battery per 100 answers, and accuracy against the desktop results
5	**A physical-laptop replication and distilled notes.** Latency was measured on laptop-class containers, and only verbatim notes were evaluated end to end (5.8.19).	One or two consumer laptops (with and without an NPU/GPU); the structured write policy already implemented	`scripts/app_latency.py` and the pilot on each laptop; gap closed and tokens with distilled vs verbatim notes
6	**Stronger evaluation basis.** Single-hop and reasoning results rest on pilot samples, and the cloud reference was sampled once (5.8.20).	n ≥ 200 per task; repeated cloud samples or a deterministic open frontier model; non-English data	Confidence intervals that exclude the pilot's uncertainty; variance across repeats
7	**A learned entity gate.** The rule-based entity gate wrongly abstained on 2.2% of HotpotQA questions whose wording names an entity differently from its paragraph (5.8.3).	A small named-entity recogniser and alias table (e.g. Wikidata aliases) in place of capitalisation rules	Abstention on answerable vs unanswerable questions; full-support recall
8	**Better routing signals.** The learned router reached 61–65% against an oracle of 79% (5.8.12).	The small model's own confidence and the grounding signal as router features	Held-out accuracy and cloud share against the same oracle
9	**Better lookups.** In the learning stream the answer was in the fetched text for only about half of the questions, and the reader was right on most of those (5.8.14): the lookup sets the ceiling.	Search with a key where the user has one; Wikidata for entity disambiguation; more than the lead section	Answer-in-fetched-text rate and accuracy on a fresh stream
10	**Sleep with real users.** The adapter could not help with a few hundred benchmark questions; a user's own approved answers are a different, narrower distribution.	Opt-in logging of the adapter's acceptance tests in the user study (priority 1)	Recall on each user's held-out repeated questions before and after sleep

Each direction should be scoped with the supervisor. The first two are the most valuable because they test the claim the application makes to its users, that it grows with them, under the conditions in which it will be used.